What Do Labeling Functions Reveal About Agent Stagnation? An Empirical Study of Weak Supervision for Sequential Agent Behavior

Anonymous

Abstract

LLM-based agents frequently exhibit *stagnation*—executing plausible but collectively unproductive action sequences during long-horizon tasks. Detecting such episodes is important for cost control and intervention design, yet labeled trajectory data remains scarce. We present the first large-scale study applying the data-programming paradigm to agent trajectory labeling, constructing a weak-supervision framework with three domain-specific labeling functions (LFs) over 54,809 windows from SWE-bench agent traces. Our study yields three principal findings, all of which are *negative* with respect to the hypothesis that a learned temporal probe can reliably detect stagnation from weak labels. First, high performance previously reported on high-confidence subsets (macro-F1 0.91 at conf ≥ 2) drops to 0.74 on the full dataset, and on human-annotated gold labels the probe (F1 0.68) *underperforms* a simple 3-repetition heuristic (F1 0.72). Second, cross-agent transfer from SWE-agent to OpenHands degrades by 12 percentage points, leaving the probe only 1pp above the repetition baseline. Third, architecture ablations (GRU, Transformer, MLP) and data-scaling experiments converge on the same ceiling, confirming label quality—not model capacity—as the bottleneck. On the positive side, temporal analysis of LF activations uncovers a reproducible *stagnation onset signature*: command repetition fires 1.8 steps before the onset, file-diff stalls concurrently, and outcome signals lag by 2.3 steps. We identify four distinct stagnation transition patterns covering 87% of observed episodes, yielding the first empirically-grounded taxonomy of agent stagnation modes. We release our framework, labels, and analysis code to support future work on agent monitoring.

1 Introduction

Large-language-model agents are increasingly deployed on complex, multi-step tasks such as software engineering [Yang et al., 2024, Wang et al., 2024], web navigation, and scientific discovery. A recurring failure mode is *stagnation*: the agent continues to act—issuing non-identical, individually plausible commands—yet makes no meaningful progress toward its goal. Unlike simple infinite loops, stagnation is *semantically diverse*: the surface actions vary while the underlying strategy remains stuck.

Reliable stagnation detection would enable early termination of wasteful trajectories, saving compute and API costs. Existing approaches fall into three camps: (i) cycle detectors that match structural repetitions but miss semantically diverse stagnation; (ii) step-level value estimators that score individual actions but lose window-level temporal dynamics; and (iii) self-reflection mechanisms that ask the stalled model to diagnose itself.

We investigate whether the *data-programming* paradigm—successful in NLP tasks such as relation extraction and sentiment analysis—can provide training signal for stagnation detection without manual annotation. We design three labeling functions (LFs) over agent trajectory windows:

`command_repetition`, `file_diff_delta`, and `outcome_backpropagation`, and train a GRU-based probe on their majority-vote labels.

Our study was originally motivated by three claims: (1) weak-supervision labels generalize beyond the high-confidence subset, (2) learned temporal probes outperform heuristic baselines, and (3) the learned signal transfers across agent implementations. **None of these claims hold at the originally hypothesized strength.** Instead, our contribution is an honest empirical accounting of *where and why* data programming fails on sequential agent data, together with a positive finding—a temporally-grounded taxonomy of stagnation onset patterns—that does not depend on probe performance.

Concretely, this paper contributes:

1. The first large-scale weak-supervision labeling framework for agent stagnation, including a taxonomy of LF temporal dynamics (Section 8).

2. A systematic characterization of the failure mode where high-confidence subset performance ($conf \geq 2$ macro-F1 0.91) does not generalize to the full label distribution (0.74) or human gold labels (0.68), quantifying the selection bias inherent in confidence-filtered data programming on sequential data (Section 4).

3. A battery of negative results—on cross-agent transfer, architecture ablations, data scaling, and early termination utility—that collectively delineate the current limits of lightweight probes trained on weak labels and point toward concrete future directions (Sections 6–10).

2 Related Work

Agent Monitoring and Intervention. Cycle detection methods identify structural repetitions in agent traces. Step-level scoring approaches assign per-action value estimates but aggregate them uniformly, losing temporal structure. Self-reflection mechanisms rely on the stalled agent's own judgment, which is unreliable precisely when the agent is stuck. Our work differs by framing stagnation as a *window-level* classification problem and studying whether weak supervision can provide the needed labels.

Data Programming and Weak Supervision. The Snorkel paradigm uses noisy labeling functions combined via a label model to create training data without manual annotation. Applications have spanned relation extraction, medical NLP, and image classification. To our knowledge, this is the first application to *sequential agent behavior data*, where LFs produce temporally-structured rather than i.i.d. outputs.

LLM-as-Judge. Recent work shows that LLMs can serve as effective evaluators. We include an LLM-as-judge baseline to contextualize the accuracy–cost tradeoff.

3 Framework and Method

3.1 Task Formulation

Given an agent trajectory $\tau = (a_1, a_2, \ldots, a_T)$, we extract overlapping windows of k consecutive steps. Each window $w_i = (a_i, \ldots, a_{i+k-1})$ is classified as PROGRESS or STAGNATION. We set $k = 5$ based on ablation (Section 9.1).

3.2 Labeling Functions

We design three LFs targeting complementary stagnation signals:

- `command_repetition`: Fires when ≥ 3 of k steps execute commands matching a configurable similarity threshold. Captures the most overt form of stagnation.
- `file_diff_delta`: Fires when the cumulative file change within the window falls below a threshold (near-zero productive output). Concurrent with stagnation onset.
- `outcome_backpropagation`: Propagates the final trajectory outcome (task pass/fail) backward into windows. A lagging indicator that fires after stagnation has been underway.

Labels are assigned by majority vote. We define the *confidence* of a window as the number of LFs that agree: $\text{conf} \in \{1, 2, 3\}$.

3.3 Probe Architecture

Each step's textual content is embedded using a frozen sentence encoder (1024-d). A single-layer GRU ($\sim$530K parameters) with step-level attention processes the k-step sequence, followed by a linear classifier. We additionally experiment with a supervised contrastive (SupCon) auxiliary loss. The architecture is deliberately simple; our focus is on the *labeling framework* rather than model design.

3.4 Data

We extract 54,809 windows from SWE-bench agent trajectories (SWE-agent). Of these, 15,401 windows have $\text{conf} \geq 2$ (28%) and 792 have $\text{conf} = 3$ (1.4%). The remaining 39,408 windows (72%) have $\text{conf} = 1$, meaning only a single LF voted, making the label equivalent to one heuristic rule's output. For cross-agent evaluation, we collect 28,340 windows from 1,847 OpenHands trajectories. A human-annotated gold set of 400 windows (3 annotators, majority vote) serves as ground truth.

4 Generalization Failure: High-Confidence Bias

Our first and most consequential finding is that performance on the $\text{conf} \geq 2$ subset—previously reported as macro-F1 0.91—does not generalize to the full dataset.

Table 1: Performance on high-confidence subset vs. full data.

Setting	Train Size	Macro-F1	Stag-F1	Prog-F1	Δ vs Repeat-3
conf$\geq$2	15,401	0.9055	0.8593	0.9518	+6.24pp
conf$\geq$1 (full)	54,809	0.74	0.67	0.81	+1.2pp
Δ	—	−16.55pp	−18.93pp	−14.18pp	−5.04pp

Stagnation-F1 suffers the largest drop (−18.93pp) because $\text{conf} = 1$ windows contain the most ambiguous stagnation cases—boundary situations where a single LF fires but the signal is unreliable. The probe's advantage over Repeat-3 shrinks from 6.24pp to 1.2pp, which is not statistically significant under bootstrap resampling ($p > 0.05$). This confirms that the $\text{conf} \geq 2$ result is an artifact of *selection bias*: high-confidence windows are the "easy" cases where multiple LFs agree on pattern-obvious stagnation.

4.1 LF Oracle and Probe Incremental Value

To test whether the probe learns anything beyond the LF rules themselves, we compare it against an LF oracle that directly applies majority vote at inference time (no learning).

Table 2: Probe vs. LF Oracle vs. baselines.

Method	conf≥2 F1	Full F1	Training Cost	Interpretability
LF Oracle (majority vote)	0.92	0.73	None	Full
GRU Probe	0.9055	0.74	0.5 GPU-hr	Low
Repeat-3	0.8432	0.728	None	Full

The LF oracle *outperforms* the probe on the conf ≥ 2 subset (0.92 vs. 0.91), confirming the probe merely fits LF outputs rather than learning signals beyond the rules. On full data, the gap is 1pp—negligible. The probe's existence is difficult to justify when a zero-cost, fully interpretable oracle matches it.

4.2 Human Gold-Label Evaluation

The most decisive evidence comes from evaluation against 400 human-annotated windows.

Table 3: All methods evaluated on human gold labels vs. LF-based labels.

Method	Gold-F1	LF-label F1	Δ
GRU Probe (conf≥2)	0.78	0.9055	−12.55pp
GRU Probe (full)	0.68	0.74	−6.0pp
Repeat-3	0.72	0.8432	−12.32pp
LF Oracle	0.65	0.92	−27.0pp
Mean-pool + LR	0.66	0.72	−6.0pp
LLM Judge (few-shot)	0.84	—	—

On gold labels, the full-data probe (F1 0.68) *underperforms* Repeat-3 (0.72) by 4pp. The LF oracle drops most dramatically (−27pp), confirming that LF labels diverge substantially from human judgment. LF majority-vote accuracy against human labels is only 67%, far below the 81.5% reported on the conf ≥ 2 subset.

Annotator Agreement. Fleiss' $\kappa = 0.52$ (moderate agreement) reveals that stagnation itself is a high-subjectivity concept. Agreement is reasonable for clear progress (κ=0.68) and clear stagnation (κ=0.61), but poor for semantic stagnation (κ=0.38) and ambiguous cases (κ=0.29).

This quantifies the inherent difficulty of the task and suggests future work should explore continuous progress scores rather than binary labels.

5 Baseline Comparisons

5.1 LLM-as-Judge

LLM-as-judge achieves 0.86 macro-F1 (few-shot), 12pp above the full-data probe. The probe retains a cost advantage (181× cheaper) and latency advantage (711× faster), but the accuracy gap is too

Table 4: Inter-annotator agreement by category.

Category	Fleiss κ	Agreement %
Overall	0.52	72%
Clear progress	0.68	84%
Clear stagnation	0.61	78%
Semantic stagnation	0.38	58%
Ambiguous	0.29	45%

Table 5: LLM-as-Judge vs. probe: accuracy, latency, and cost.

Method	Macro-F1	Stag-F1	Latency (ms)	Cost/1k ($)	Training
GPT-4o zero-shot	0.82	0.76	3200	14.50	No
GPT-4o few-shot (5)	0.86	0.80	4100	18.20	No
Claude few-shot (5)	0.84	0.78	2800	12.80	No
GRU Probe (full)	0.74	0.67	4.5	0.08	Yes
GRU Probe (conf$\geq$2)	0.9055	0.8593	4.5	0.08	Yes

large to position the probe as a general replacement. On gold labels, the LLM judge leads all methods with F1 0.84.

Table 6: Cost–accuracy Pareto analysis (annual cost at 50k windows).

Method	Macro-F1	Annual Cost	Real-time Feasible
GRU Probe	0.74	$4	Yes
LF Oracle	0.73	$0	Yes
Repeat-3	0.728	$0	Yes
LLM Judge	0.86	$910	No

5.2 Mean-Pooling + Logistic Regression

The 2pp gap on full data is not statistically significant ($p \approx 0.08$–0.15 under bootstrap), directly challenging the necessity of temporal modeling. Mean-pool + LR achieves comparable performance with 1/530 the parameters and 1/140 the training time, suggesting the bottleneck lies in label quality rather than model capacity.

5.3 Non-NLP Structural Features

A gradient-boosted model on 12 structural features (line changes, AST diffs, test counts) reaches 0.70 full-data F1. NL embeddings provide only a 4pp increment, which is non-trivial but not decisive—structural features alone capture most of the stagnation signal.

6 Cross-Agent Transfer

Transfer drops performance by 12pp ($0.74 \rightarrow 0.62$), and the probe barely edges out Repeat-3 by 1pp. Stagnation-F1 is particularly poor (0.53), indicating the probe cannot recognize OpenHands-specific

Table 7: Temporal (GRU) vs. static (mean-pool + LR) aggregation.

Method	**conf≥2 F1**	**Full F1**	**Params**	**Train Time**
GRU Probe	0.9055	0.74	∼530K	28 min
Mean-pool + LR	0.87	0.72	∼1K	12 sec
Gap	+3.55pp	+2.0pp	530×	140×

Table 8: NLP embedding features vs. structural-only features.

Method	**Features**	**conf≥2 F1**	**Full F1**
GRU Probe (NLP)	1024-d embeddings	0.9055	0.74
GBM (Non-NLP)	12 structural features	0.83	0.70
Gap	—	+7.55pp	+4.0pp

stagnation patterns, which differ in command structure (more Python API calls), trajectory length distribution (shorter, with faster failure modes), and embedding distribution.

Despite adapting all three LFs, overall coverage drops to 0.78 (from 1.0 on SWE-agent) with lower estimated accuracy per LF. The conf ≥ 2 ratio is only 22% (vs. 28% on SWE-agent), amplifying the noisy-label problem. These results indicate that the LF design, not the probe architecture, is the primary bottleneck for cross-agent generalization.

7 Error Analysis

71% of errors concentrate in conf $= 1$ windows, with an error rate of 0.31—more than 3× the rate for conf $= 2$. This quantifies the selection bias: the probe performs well on easy, high-agreement cases and poorly on the ambiguous majority.

Semantic stagnation accounts for 58% of false negatives. These are cases where the agent varies its commands but pursues a fundamentally unproductive strategy—precisely the pattern that neither rule-based LFs nor the probe can capture with current embeddings. Among false positives, 42% involve exploratory behavior misclassified as stagnation.

8 LF Temporal Dynamics and Stagnation Taxonomy

This section presents our primary positive finding, which is independent of probe performance.

8.1 Temporal Ordering of LF Activations

The three LFs exhibit a consistent temporal ordering in 72% of stagnation episodes: `command_repetition` fires first (leading by 1.8 steps on average), `file_diff_delta` activates near-simultaneously, and `outcome_backpropagation` trails by 2.3 steps. This leading→concurrent→lagging structure constitutes a reproducible *stagnation onset signature*.

8.2 Stagnation Transition Patterns

Analysis of LF activation sequences reveals four distinct transition patterns:

Table 9: Cross-agent transfer: SWE-agent → OpenHands (zero-shot).

Setting	Macro-F1	Stag-F1	Prog-F1	vs Repeat-3
SWE-agent (conf≥2, in-domain)	0.9055	0.8593	0.9518	+6.24pp
SWE-agent (full, in-domain)	0.74	0.67	0.81	+1.2pp
OpenHands (zero-shot transfer)	0.62	0.53	0.71	+1.0pp
OpenHands Repeat-3	0.61	0.52	0.70	—

Table 10: LF adaptation coverage for OpenHands.

LF	Adapted	Coverage	Accuracy Est.
command_repetition	Yes (regex)	0.82	∼0.65
file_diff_delta	Yes (API)	0.71	∼0.60
outcome_backprop	Yes (direct)	0.81	∼0.70
Overall coverage	—	0.78	—

These four types cover 87% of observed episodes. Type A (classic repetition) is the most common and easiest to detect. Type B (exploration failure) is notable: the agent tries different files but encounters the same errors, requiring semantic understanding to diagnose. Type D (cascading failure) is the most insidious, as the root cause precedes observable stagnation signals.

8.3 LF Marginal Contributions

Marginal contributions follow the temporal ordering: the leading indicator (`command_repetition`) contributes most (−4.0pp on full data when removed), while the lagging indicator (`outcome_backprop`) contributes least (−1.0pp). This ranking has practical implications: in deployment, monitoring `command_repetition` alone may suffice as an early warning signal.

9 Ablation Studies

9.1 Window Size

Performance varies within a 3pp range; $k=5$ is mildly optimal. The low sensitivity indicates window size is not a critical design parameter.

9.2 Architecture

All architectures converge within a 2pp range on full data. Even the simplest model (MLP, 260K params, 8 min training) achieves 0.72, only 2pp below the best. This confirms the performance ceiling is set by label quality, not model capacity.

9.3 Supervised Contrastive Loss

SupCon provides a marginal 1pp improvement on full data, consistent with the hypothesis that noisy labels undermine contrastive learning: the loss simultaneously pushes representations toward correct and incorrect label-mates.

Table 11: Error distribution stratified by LF confidence level.

Confidence	Windows	Errors	Error Rate	% of Total Errors
conf=1	39,408	12,216	0.31	71%
conf=2	14,609	1,315	0.09	26%
conf=3	792	24	0.03	3%

Table 12: False negative decomposition.

FN Type	Count	Pct	Example Pattern
Semantic stagnation	71	58%	Different commands, same ineffective strategy
Late-onset stagnation	30	24%	Progress in first 2 steps, stall in last 3
Ambiguous boundary	22	18%	Partial progress, debatable label

9.4 Data Scaling

The learning curve saturates around 30% of the data. Going from 50% to 100% yields only 1pp, indicating that additional weakly-labeled data provides diminishing returns—the ceiling is imposed by label noise, not data scarcity.

9.5 Representation Quality

GRU hidden states show poor cluster separation (silhouette 0.28) and high class overlap (35%), reflecting both label noise and the model's inability to learn a clear decision boundary. Attention weights show partial concentration at stagnation onset positions (0.45) but are not sharp enough to support strong interpretability claims.

10 Application: Simulated Early Termination

The probe saves 18% of tokens but at the cost of 4.5pp in task success rate, yielding an efficiency ratio of 4.0. Repeat-3 achieves a *better* efficiency ratio (4.2) by saving slightly fewer tokens (16%) with a smaller success-rate penalty (−3.8pp). The intervention paradox rate of 0.35 means that 35% of terminated trajectories might have succeeded if allowed to continue. The probe's practical value for early termination is marginal at best relative to simple heuristics.

11 Discussion

What Went Wrong. The core failure is that data programming on sequential agent data produces a bimodal quality distribution: high-confidence labels (conf ≥ 2) are reliable but cover only 28% of windows, while the remaining 72% (conf $= 1$) carry noise rates exceeding 30%. This violates the i.i.d. assumption underlying standard data programming, where each labeling function provides independent, domain-relevant signal. In sequential data, LFs produce temporally correlated outputs—if command repetition fires, file-diff stall is likely to fire in the next step—reducing effective independence and inflating confidence estimates.

What Went Right. The LF temporal dynamics analysis (Section 8) succeeds precisely because it does not depend on label quality. The four stagnation transition patterns constitute the first

Table 13: LF activation timing relative to stagnation onset.

LF	Avg Lead/Lag (steps)	Role	Activation at Onset
command_repetition	−1.8 (lead)	Leading	0.78
file_diff_delta	−0.5 (concurrent)	Concurrent	0.71
outcome_backprop	+2.3 (lag)	Lagging	0.64

Table 14: Stagnation onset transition patterns.

Pattern	Freq.	Description
Type A: cmd→file→outcome	42%	Classic repetition-driven stagnation
Type B: file→cmd→outcome	18%	Exploration failure (new files, same errors)
Type C: cmd only	15%	Pure command loop, files never change
Type D: outcome→file→cmd	12%	Cascading failure from bad early decision

empirically-grounded taxonomy of how agents enter stagnation states. The leading/concurrent/lagging ordering of LFs has direct deployment value: monitoring only `command_repetition` provides 1.8 steps of advance warning.

When Is the Probe Useful? In a narrow regime—cost-constrained, real-time monitoring where the 12pp accuracy gap versus LLM-as-judge is acceptable ($4/year vs. $910/year at 50k windows)—the probe or even the zero-cost LF oracle may be reasonable. However, for applications requiring high precision (e.g., automated intervention), the LLM-as-judge is strictly superior.

12 Limitations

- **Domain scope.** All data comes from software engineering agents (SWE-agent, OpenHands). Transfer to non-SE domains (web navigation, scientific discovery) is untested and likely to require entirely new LFs.
- **Binary labels.** Human annotator disagreement ($\kappa = 0.52$) suggests stagnation is better modeled as a continuous spectrum than a binary classification.
- **Gold set size.** The 400-window gold set, while sufficient to reveal trends, is small for fine-grained subgroup analysis.
- **LF coverage.** We use three LFs; richer LF sets (e.g., incorporating error message semantics or API call patterns) may improve conf$\geq$2 coverage.
- **Embedding quality.** We use frozen sentence encoders; domain-adapted embeddings might improve the probe's ability to distinguish semantic stagnation from genuine exploration.

13 Conclusion

We set out to validate weak supervision as a practical approach to agent stagnation detection and found that—on the full data distribution—it does not deliver on the promise shown by high-confidence subsets. The learned probe does not reliably outperform simple heuristics, does not

Table 15: Hold-one-out ablation of labeling functions.

Held-out LF	**conf≥2 F1**	**Δ (pp)**	**Full F1**	**Δ (pp)**
None (all LFs)	0.9055	—	0.74	—
− command_repetition	0.8750	−3.05	0.70	−4.0
− file_diff_delta	0.8830	−2.25	0.71	−3.0
− outcome_backprop	0.8920	−1.35	0.73	−1.0

Table 16: Window size ablation.

Window k	**conf≥2 F1**	**Full F1**	**vs Repeat-3 (full)**
$k=3$	0.88	0.71	−0.18pp
$k=5$ (default)	0.9055	0.74	+1.2pp
$k=7$	0.90	0.73	+1.0pp

transfer across agent implementations, and does not offer compelling advantages over LLM-based judges in accuracy.

However, the attempt was not without value. We contribute the first large-scale weak-supervision framework for agent trajectory labeling, demonstrate a concrete failure mode of data programming on sequential data (confidence-filtered selection bias), and discover a temporally-structured stagnation taxonomy with four distinct onset patterns. The temporal ordering of labeling functions—leading, concurrent, and lagging indicators—provides actionable engineering insight independent of any learned model.

These results point to two future directions: (1) improving label quality through hybrid approaches (e.g., using LLM judges to label the conf $= 1$ subset) and (2) designing agent-agnostic labeling functions or meta-learning adaptation strategies for cross-agent transfer. We release our framework and data to enable both lines of inquiry.

References

J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Xia, K. Narasimhan, and O. Press. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

X. Wang, Y. Li, C. Chen, et al. OpenDevin: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré. Snorkel: Rapid training data creation with weak supervision. In *VLDB*, 2017.

L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS*, 2024.

P. Khosla, P. Teterwak, C. Wang, et al. Supervised contrastive learning. In *NeurIPS*, 2020.

Table 17: Architecture ablation.

Architecture	Params	conf≥2 F1	Full F1	Train Time
GRU + Attention	∼530K	0.9055	0.74	28 min
GRU + Mean-pool	∼520K	0.8950	0.73	25 min
Transformer (2L)	∼610K	0.8980	0.74	35 min
MLP on pooled	∼260K	0.8820	0.72	8 min

Table 18: Effect of supervised contrastive (SupCon) auxiliary loss.

Setting	conf≥2 F1	Full F1	Embed Separation
With SupCon	0.9055	0.74	0.42
Without SupCon	0.8980	0.73	0.38
Δ	+0.75pp	+1.0pp	+0.04

Table 19: Data scaling curve (full-data regime).

Train %	Windows	Macro-F1	Δ vs previous
10%	∼4,380	0.69	—
25%	∼10,960	0.72	+3pp
50%	∼21,920	0.73	+1pp
100%	43,847	0.74	+1pp

Table 20: Representation space quality metrics.

Metric	Value	Ideal
Silhouette score	0.28	>0.5
Class overlap %	35%	<15%
Attention onset concentration	0.45	>0.7

Table 21: Simulated early termination on held-out trajectories.

Detector	Tokens Saved (%)	Success Rate Drop (pp)	Efficiency Ratio
GRU Probe (full)	18%	−4.5	4.0
Repeat-3	16%	−3.8	4.2
LF Oracle	22%	−2.1	10.5
LLM Judge	25%	−1.5	16.7